Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский Нижегородский государственный университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Арифметические операции с полиномами»

Выполнил: студент группы 3824Б1ФИ2 Ненев Алексей Евгеньевич
Проверил: Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3.Руководство программиста.....8
3.1. Описание структуры программы 8
3.2. Описание структур данных 9
3.3. Описание алгоритмов 14
4. Результаты экспериментов и тестирования 17
Заключение 19
Литература 20

Введение

Полином – это алгебраическое выражение, представляющее собой сумму или разность конечного числа мономов (произведений чисел и переменных в неотрицательных степенях). Это фундаментальное понятие алгебры, используемое для описания функциональных зависимостей, аппроксимации функций и решения уравнений. Полиномы широко используются в науке и это обуславливает потребность в инструментах для более удобной и быстрой работы с полиномами.

Эта работа представляет из себя реализацию полинома на языке C++. Класс полинома реализован на односвязном списке, где в каждой ячейке хранятся мономы – коэффициент и степени переменных. Также для удобной работы реализованы базовые операции с полиномами.

1. Постановка задачи

Цель данной работы – реализовать программу на языке программирования C++, которая реализует для полиномов переменных X, Y, Z следующие операции:

1. Сложение/вычитание полиномов
2. Умножение двух полиномов
3. Умножение полинома на константу
4. Ввод/вывод полиномов

Особенности реализации:

1) В качестве структуры хранения полинома используется односвязный список мономов с ненулевыми коэффициентами

2) Степень полинома хранится в "свернутом" виде, т.е. степень представлена как трехзначное число, где число сотен –это степень при переменной “x”, число десятков -степень при переменной “y”, число единиц -степень при переменной “z”.

3) Сложение полиномов осуществляется алгоритмом слияния упорядоченных массивов.

4) Вычитание полиномов выполняется через сложение с умножением на константу (C = A -B = A + (-1) * B)

5) При умножении и сложении(вычитании) в итоговом полиноме приведены подобные слагаемые и не хранятся мономы с нулевыми коэффициентами.

6) Если при умножении полиномов полученные степени переменных больше 9, выводится сообщение об ошибке.

2. Руководство пользователя

Приложение работает в режиме командной строки. После выполнения каждой команды (кроме "Выход"), вы снова увидите запрос "Enter command: ", ожидающий новую команду.

1 - Ввести полином: Добавить новый полином в хранилище приложения.

2 - Вывести полином: Отобразить полином по его ID.

3 - Сложить два полинома: Выполнить сложение двух хранимых полиномов.

4 - Вычесть второй полином из первого: Выполнить вычитание двух хранимых полиномов.

5 - Умножить два полинома: Выполнить умножение двух хранимых полиномов.

6 - Умножить полином на константу: Умножить выбранный полином на числовое значение.

7 - Вывести все полиномы: Отобразить все полиномы, хранящиеся в приложении, вместе с их ID.

8 - Выход: Завершить работу приложения.

9

Каждый моном в полиноме задается парой "коэффициент степень". Общий формат: C1 D1 C2 D2 C3 D3 ...

C - это коэффициент (может быть целым или десятичным числом, например,
5, 12.3, -2.5).

D - это степень, трёхзначное целое число, определяющее степени переменных x, y, z.

Первая цифра (сотни) - степень переменной x.

Вторая цифра (десятки) - степень переменной y.

Третья цифра (единицы) - степень переменной z.

Ограничение: Каждая степень (для x, y, z) не может быть больше 9. Соответственно, число D должно быть в диапазоне от 000 до 999.

Краткий обзор команд

1 - Ввести полином Действие: Добавляет новый полином в хранилище приложения.

Ввод: Приложение попросит "Enter polynomial for example (12 111 3.7 234) = 12xyz + 3.7x^2 y^3 z^4: ". Введите ваш полином в указанном формате.

Обработка: Приложение парсит введенную строку, создает объект Polinomial, упрощает его (объединяет подобные члены, удаляет нулевые коэффициенты) и сохраняет.

Вывод: "Polynomial added, its ID is N." где N - это уникальный идентификатор (ID) только что введенного полинома.

2 - Вывести полином

Действие: Отображает ранее введенный полином.

Ввод: Приложение попросит "Enter the polinomial's ID". Введите ID полинома, который вы хотите вывести (например, 1).

Обработка: Приложение ищет полином по указанному ID.

Вывод: "Polynomial N: [Форматированное представление полинома]"

3 - Сложить два полинома

Действие: Вычисляет сумму двух полиномов. Результат выводится на экран, но не сохраняется.

Ввод: Приложение попросит "Enter the polinomials' IDs". Введите два ID полиномов, которые вы хотите сложить, разделенные пробелом (например, 1 2).

Обработка: Приложение извлекает два полинома по их ID, выполняет операцию сложения и упрощает результат.

Вывод: "Result of addition: [Форматированное представление результирующего полинома]"

4 - Вычесть второй полином из первого

Действие: Вычисляет разность двух полиномов (первый - второй). Результат выводится на экран, но не сохраняется.

Ввод: Приложение попросит "Enter the polinomials' IDs". Введите ID первого полинома, затем ID второго полинома, разделенные пробелом (например, 1 2).

Обработка: Приложение извлекает полиномы, выполняет вычитание и упрощает результат.

Вывод: "Result of subtraction: [Форматированное представление результирующего полинома]"

5 - Умножить два полинома

Действие: Вычисляет произведение двух полиномов. Результат выводится на экран, но не сохраняется.

Ввод: Приложение попросит "Enter the polinomials' IDs". Введите ID первого полинома, затем ID второго полинома, разделенные пробелом (например, 1 2).

Обработка: Приложение извлекает полиномы, выполняет умножение каждого члена первого полинома на каждый член второго, суммирует степени и умножает коэффициенты, затем упрощает результат.

Важно: Если при умножении степеней x, y или z их сумма превысит 9, приложение выбросит ошибку runtime_error и завершится. Это связано с ограничением на трехзначное представление степеней.

Вывод: "Result of multiplication: [Форматированное представление результирующего полинома]

6 - Умножить полином на константу

Действие: Умножает выбранный полином на числовое значение. Результат выводится на экран, но не сохраняется.

Ввод: Приложение попросит "Enter the polinomial's ID", затем "Enter constant:". Введите ID полинома, затем константу (например, 1 и -2.5).

Обработка: Приложение извлекает полином, умножает каждый его коэффициент на введенную константу.

Вывод: "Result of multiplication by constant: [Форматированное представление результирующего полинома]"

7 - Вывести все полиномы

Действие: Отображает все полиномы, которые были введены и хранятся в приложении, вместе с их присвоенными ID.

Ввод: Просто введите 7.

Обработка: Приложение итерирует по своему хранилищу и выводит каждый полином.

Вывод: Список всех полиномов в формате "ID N: [Форматированное представление полинома]".

8 - Выход

Действие: Завершает работу приложения.

Ввод: Просто введите 8.

Вывод: "Turned off"

Обработка ошибок

Некорректный ID: Если вы введете несуществующий ID полинома (например, 0 или 5, когда есть только 2 полинома), приложение выведет сообщение "Invalid polynomial ID. Please enter a number between 1 and N." и попросит ввести ID снова.

Нечисловой ввод: Если вы введете текст вместо числа (например, abc вместо 1 для команды или ID), приложение выведет "Invalid input. Please enter a valid number." и попросит повторить ввод.

Пустое хранилище: Если вы попытаетесь выполнить операцию (кроме ввода), когда нет ни одного полинома, приложение сообщит "No polynomials have been entered yet, please add a polynomial first (command 1)."

Превышение степени: Если в результате операции умножения степень любой из переменных (x, y, z) превысит 9, приложение немедленно завершит работу с сообщением об ошибке runtime_error (например, "Degrees over 9 are not allowed...").

3. Руководство программиста

3.1 Общее описание программы

Программа реализует функциональность для работы с полиномами от трех переменных (x, y, z), где каждая переменная может иметь степень от 0 до 9. Полиномы хранятся в пользовательской реализации односвязного списка и поддерживают базовые арифметические операции: сложение, вычитание, умножение (полином на полином, полином на константу). Предоставляется консольное приложение для взаимодействия с пользователем.

3.2 Структура программы и назначение модулей

1.Программа логически делится на два основных модуля:

Файл Polinomial: Реализует основные структуры данных и логику работы с полиномами.

Приложение App: Предоставляет консольный интерфейс пользователя для взаимодействия с библиотекой.

2.1. Модуль "Polinomial"

Назначение: Предоставляет ядро функциональности для создания, хранения, манипулирования и выполнения арифметических операций над полиномами.

Состоит из:

Заголовочный файл Header.h : Содержит объявления всех классов, структур, итераторов, а также глобальные константы. Определяет интерфейс библиотеки.

Файл реализации Polinomial.cpp: Содержит определения всех методов классов и дружественных функций, объявленных в заголовочном файле.

2.2. Модуль "Приложение App"

Назначение: Обеспечивает интерактивный консольный интерфейс для пользователя. Принимает команды пользователя, вызывает соответствующие функции из библиотеки Polinomial и отображает результаты.

Состав:

Файл main.cpp: Содержит главную функцию программы, меню команд, логику обработки пользовательского ввода и вывода.

Вспомогательные функции: print_menu(), get_input(), get_polynomial_id().

3.2 Подробное описание классов, структур и глобальных переменных

3.2.1. Структура Monomial

Назначение: Представляет собой один член полинома (моном).

Члены данных:

double Coefficient: Числовой коэффициент монома.

int Degrees: Целое число, кодирующее степени переменных x, y, z.

Формат: XYZ, где X (сотни) — степень x, Y (десятки) — степень y, Z (единицы) — степень z. Каждая степень X, Y, Z находится в диапазоне [0, 9].

Конструкторы:

Monomial(): Конструктор по умолчанию. Инициализирует коэффициент и степень нулями.

Monomial(double C, int D): Конструктор с параметрами. Инициализирует коэффициент C и степень D. Перегруженные операторы:

bool operator<(const Monomial& other) const: Определяет порядок мономов для сортировки. Сортирует мономы по убыванию степени (Degrees > other.Degrees). Используется my_list::sort().

bool operator==(const Monomial& other) const: Сравнивает два монома на равенство. Считаются равными, если их Degrees одинаковы и их Coefficients тоже одинаковы

bool operator!=(const Monomial& other) const: Обратная операция к operator==.

3.2.2 Структура Node

Назначение: Базовый строительный блок для my_list, представляющий узел в односвязном списке.

Члены данных:

Monomial Elem: Сам моном, который хранится в этом узле.

Node* next: Указатель на следующий узел в списке. nullptr для последнего узла.

Конструктор:

Node(Monomial M): Инициализирует Elem переданным мономом M и next как nullptr.

Деструктор:

~Node() = default: Деструктор по умолчанию.

3.2.3. Класс my_list

Назначение: Пользовательская реализация односвязного списка, предназначенная для хранения объектов Monomial. Заменяет std::list.

Члены данных (приватные):

Node* head: Указатель на первый узел списка. nullptr, если список пуст.

Node* tail: Указатель на последний узел списка. nullptr, если список пуст.

std::size_t _size: Хранит текущее количество элементов в списке.

Вложенные классы: my_list::iterator:

Назначение: Предоставляет изменяемый доступ к элементам Monomial в списке my_list.

Члены данных (приватные):

Node* current_node: Указатель на текущий Node, на который указывает итератор.

Конструктор:

explicit iterator(Node* ptr = nullptr): Инициализирует current_node.

Перегруженные операторы:

Monomial& operator*() const: Возвращает ссылку на Monomial текущего узла. Позволяет изменять элемент.

Monomial* operator->() const: Возвращает указатель на Monomial текущего узла. Позволяет обращаться к членам Monomial через ->.

iterator& operator++(): Префиксный инкремент. Перемещает итератор к следующему узлу. Возвращает ссылку на сам итератор.

bool operator==(const iterator& other) const: Сравнивает два итератора, проверяя, указывают ли они на один и тот же Node.
bool operator!=(const iterator& other) const: Обратная операция к operator==.
my_list::const_iterator:

Назначение: Предоставляет константный доступ к элементам Monomial в списке my_list. Используется для константных объектов my_list или в const методах, где элементы не должны изменяться.

Члены данных (приватные):

const Node* current_node Указатель на константный Node

Конструктор:

explicit const_iterator(const Node* ptr = nullptr): Инициализирует current_node.

Перегруженные операторы:

const Monomial& operator*() const: Возвращает константную ссылку на Monomial текущего узла. Не позволяет изменять элемент.
const Monomial* operator->() const: Возвращает константный указатель на Monomial текущего узла.

const_iterator& operator++(): Префиксный инкремент. Перемещает итератор к следующему узлу.
bool operator==(const const_iterator& other) const: Сравнивает два константных итератора.
bool operator!=(const const_iterator& other) const: Обратная операция к operator==.
Конструкторы:

my_list(): Конструктор по умолчанию. Создает пустой список.

my_list(const my_list& other): Конструктор копирования. Выполняет глубокое копирование: создает новые узлы для каждого элемента other и добавляет их в текущий список.
Деструктор:

~my_list(): Освобождает всю память, занятую узлами списка, вызывая clear().
Перегруженные операторы:

my_list& operator=(my_list other):

Оператор присваивания (идиома "copy-and-swap"). Принимает параметр по значению (что вызывает конструктор копирования). Затем обменивается внутренними ресурсами (head, tail, _size) с временной копией other, позволяя деструктору other освободить старые ресурсы текущего объекта.

Методы:

1) **bool empty() const:** Проверяет, пуст ли список.

2) **std::size_t size() const:** Возвращает количество элементов в списке.

3) **void clear():** Удаляет все узлы из списка и освобождает выделенную память.

4) **void emplace_back(Monomial M):** Добавляет новый моном в конец списка.

5) **void emplace_back(double c, int d):** Перегрузка emplace_back для удобного добавления монома по коэффициенту и степени.

6) **iterator begin():** Возвращает итератор, указывающий на первый элемент списка.

7) **iterator end():** Возвращает итератор, указывающий на позицию "за концом" списка (nullptr).

8) **const_iterator begin() const:**Константная версия begin

9) **const_iterator end() const:** Константная версия end().

10) **void sort():** Сортирует элементы списка пузырьком с оптимизацией.

3.2.4. Класс Polinomial

Назначение: Представляет собой полином и предоставляет высокоуровневые операции для работы с ним. Члены данных (приватные):

my_list Data: Пользовательский список, хранящий мономы полинома.

Конструкторы:

1. **Polinomial():** Конструктор по умолчанию. Создает пустой полином.
2. **Polinomial(const Polinomial& other):** Конструктор копирования. Использует конструктор копирования my_list для глубокого копирования данных.
3. **Polinomial(const std::string s):** Конструктор из строки. Парсит строку, содержащую пары "коэффициент степень", и создает полином. Автоматически вызывает simplify().

Перегруженные операторы:

1. **bool operator==(const Polinomial& other) const:** Сравнивает два полинома на равенство, проверяя размер и попарное равенство мономов с помощью итераторов.
2. **bool operator!=(const Polinomial& other) const:** Обратная операция к operator==.
3. **Polinomial& operator=(const Polinomial& other):** Оператор присваивания. Использует оператор присваивания my_list для глубокого копирования данных.
4. **Polinomial operator+(const Polinomial& other) const:** Бинарный оператор сложения. Возвращает новый полином, являющийся суммой *this и other.
5. **Polinomial operator-(const Polinomial& other) const:** Бинарный оператор вычитания. Возвращает новый полином, являющийся разностью *this и other.
6. **Polinomial operator*(const Polinomial& other) const:** Бинарный оператор умножения. Возвращает новый полином, являющийся произведением *this и other.
7. **Polinomial operator*(double Mult) const:** Бинарный оператор умножения на константу. Возвращает новый полином.
8. **friend std::ostream& operator<<(std::ostream& out, const Polinomial& P):** Дружественная функция для форматированного вывода полинома в поток.
9. **friend std::istream& operator>>(std::istream& in, Polinomial& P):** Дружественная функция для ввода полинома из потока, используя строковый конструктор.

Методы (приватные):

1. **void simplify():** Вспомогательный метод. Сортирует мономы по степени, объединяет подобные члены и удаляет мономы с коэффициентами, близкими к нулю. Поддерживает полином в каноническом виде.

3.3. Подробное описание алгоритмов

3.3.1. my_list::sort() (Сортировка списка)

Назначение: Сортирует мономы в списке my_list по убыванию степени.

Алгоритм:

Выполняет сортировку пузырьком, меняя элементы, если они не стоят в отсортированном порядке.

Сложность: O(N^2), где N — количество мономов

3.3.2. Polinomial::simplify() (Упрощение полинома)

Приводит полином к каноническому виду: сортирует мономы, объединяет подобные члены (с одинаковой степенью) и удаляет члены с нулевыми коэффициентами.

Алгоритм:

Вызывает Data.sort(), чтобы все мономы были отсортированы по степени (по убыванию).

Создает новый пустой список. Итерирует по отсортированному Data:

Берет первый моном (currentCoeff, currentDegree).

Сравнивает его со следующими мономами. Если у следующего монома та же степень, складывает его коэффициент с currentCoeff и переходит к следующему.

Продолжает, пока не встретит моном с другой степенью или не достигнет конца списка.

Сложность: O(N^2) из-за Data.sort(). Итерация и объединение после сортировки занимают O(N).

3.3.3. Polinomial::Polinomial(const std::string s) (Конструктор из строки)

Создает полином из строкового представления.

Алгоритм:

Использует std::istringstream для чтения пар "коэффициент степень" из входной строки.

Проверяет, что degree находится в допустимом диапазоне [0, 999]. В случае ошибки выбрасывает ошибку. Также игнорирует мономы с коэффициентом, близким к нулю.

Сохраняет все успешно распаршенные мономы во временный вектор

Добавляет все мономы из tempMonomials в Data с помощью emplace_back().

Вызывает simplify() для сортировки и объединения подобных членов. Сложность: Зависит от длины строки S и количества мономов N. Парсинг O(S). Заполнение Data и simplify() — O(N^2).

3.3.4. Polinomial::operator*(const Polinomial& other) (Умножение полиномов)

Назначение: Умножает текущий полином на другой полином (other)

Алгоритм:

Создает новый пустой лист для хранения мономов-результатов умножения.

Использует вложенные циклы для итерации по каждому моному, для каждой пары мономов извлекает степени x, y, z для elem1 и elem2 из их Degrees, складывает соответствующие степени.

Проверяет на превышение степени: Если какая-либо из новых степеней больше 9, выбрасывает ошибку.

Собирает новую трехзначную степень

Перемножает коэффициенты

Добавляет новый моном

Вызывает simplify() для объединения подобных членов, которые могли образоваться после умножения.

Сложность: O(N*M) для перемножения мономов (где N и M — количество мономов в полиномах) плюс O((N*M) log (N^2)) для simplify().

3.3.5 Polinomial operator+(const Polinomial& other) const; (Сложение полиномов)

Назначение: Складывает текущий полином с другим полиномом (other)

Алгоритм:

Переводит оба полинома в векторы

Объединяет оба вектора в один

Переводит полученный вектор в полином и упрощает его, убирая малые мономы и объединяя подобные

Сложность: O(N^2) из-за simplify()

4. Результаты экспериментов и тестирования

1. Тестирование Monomial (TestNameMonomial)

Проверка базовых операторов сравнения (<, ==, !=) для отдельных членов полинома (Monomial), включая особенности сортировки по убыванию степени и сравнение double с допуском.

Вывод: Операторы сравнения Monomial функционируют корректно.

2. Тестирование my_list (TestNameList)

Проверка базовых операций пользовательского списка my_list: инициализация (пустой список), добавление элементов (emplace_back), и оператор присваивания (=). Тест подтверждает отсутствие исключений при этих операциях. Вывод: Основные операции my_list работают без ошибок.

3. Тестирование инициализации Polinomial (TestNameInitialisation)

Проверка различных способов создания полиномов: из корректных строк, по умолчанию, и с использованием конструктора копирования. Особое внимание уделено проверке генерации исключений (runtime_error) при попытке создания полинома с некорректными значениями степеней (выходящими за пределы [0, 999]).

Вывод: Конструкторы Polinomial корректно обрабатывают как правильные данные, так и ошибки ввода степеней.

4. Тестирование операторов равенства Polinomial (TestNameOperatorsEquality)

Проверка операторов == и != для класса Polinomial. Включает сравнение идентичных полиномов (созданных разными способами), разных полиномов и проверку работы функции simplify() путем сравнения полинома с его "неупрощенной" версией.

Вывод: Операторы сравнения Polinomial работают правильно, учитывая внутреннее упрощение полиномов.

5. Тестирование оператора присваивания Polinomial (TestNameOperatorsEquation)

Логика: Проверка корректности оператора присваивания = и его взаимодействия с конструктором копирования, подтверждая создание глубоких копий полиномов.

Вывод: Оператор присваивания Polinomial функционирует как ожидалось, создавая независимые копии.

6. Тестирование сложения и вычитания Polinomial (TestNameOperatorsAdditionSubtraction)

Проверка операторов + и - с использованием различных комбинаций полиномов, включая те, что требуют объединения подобных членов. Тест также подтверждает алгебраические свойства (например, A+B-B = A и A-A = 0).

Вывод: Операции сложения и вычитания Polinomial реализованы корректно.

7. Тестирование умножения Polinomial на константу (TestNameOperatorsMultiplicationByConstant)

Проверка оператора умножения * с double константой. Тест подтверждает, что умножение на константу правильно масштабирует коэффициенты и сохраняет алгебраические свойства (например, A+A = A*2).

Вывод: Умножение Polinomial на константу работает правильно.

8. Тестирование умножения полиномов (TestNameMultiplicationOfPolinomials)

Проверка оператора умножения * для двух полиномов. Включает проверку корректности вычислений и особо отмечает обработку исключений (runtime_error) при превышении допустимой степени (больше 9 для любой переменной) в результате умножения.

Вывод: Умножение полиномов функционирует правильно, включая механизм защиты от превышения максимально допустимых степеней.

9. Тестирование ввода Polinomial (TestNamePolinomialInput)

Проверка оператора ввода operator>> для Polinomial с использованием std::istringstream для симуляции ввода из строки. Тест подтверждает, что полиномы, прочитанные из строкового потока, идентичны полиномам, созданным напрямую из той же строки.

Вывод: Оператор ввода Polinomial реализован корректно.

Заключение

В ходе выполнения лабораторной работы достигнуты следующие результаты:

1) Реализован класс списка для удобного хранения полинома
2) Реализована структура монома для простого и понятного представления полинома
3) Реализован класс список, который используется для хранения полинома
4) Реализован класс полинома, а также базовые операции для него
5) Реализован пользовательский интерфейс для удобной работы с полиномами

Итого, программа соответствует изначальным задачам.

Литература

1. Сайт cppreference: https://en.cppreference.com/